Разработка ядра библиотеки обнаружения аномалий OmniAD: самоописание API, потоковое обучение, сериализация моделей и инфраструктура тестирования

Отчет о научно-исследовательской работе за семестр

11 октября 2026 г.

Цель работы

Реализовать и всесторонне протестировать три взаимосвязанные архитектурные подсистемы ядра библиотеки OmniAD: механизмы самоописания алгоритмов (*discoverability*), поддержку потокового (*streaming*) обучения и унифицированную кросс-бэкендовую сериализацию моделей; построить автоматизированную многоуровневую систему тестирования, подтверждающую корректность реализованных механизмов на четырех поддерживаемых доменах данных, а также подготовить архитектурную спецификацию подсистемы бенчмаркинга как основу для дальнейшей разработки.

Аннотация

Библиотеки обнаружения аномалий, построенные на разных вычислительных фреймворках (scikit-learn, PyTorch, HuggingFace Transformers, River), исторически несовместимы между собой: они используют противоположные соглашения о знаке скора аномальности, предоставляют различные сигнатуры методов обучения и инференса, а гибридные конвейеры (например, векторизатор текста поверх классического детектора) не имеют стандартного протокола совместного сохранения. Вследствие этого разработчику приходится писать объемный связующий код под каждую комбинацию «алгоритм — домен данных», что замедляет разработку и повышает риск скрытых дефектов при переносе моделей в эксплуатацию.

Библиотека OmniAD призвана устранить данную фрагментацию, предоставляя унифицированный API поверх гетерогенных бэкендов. Однако для полноценного применения единый API должен обладать свойствами самоописания, поддерживать потоковые сценарии и обеспечивать безопасную сериализацию и десериализацию моделей. Данная работа посвящена проектированию и программной реализации указанных подсистем, устранению выявленных дефектов ядра, созданию контрактного набора тестов и формированию спецификации измерительного модуля библиотеки.

Практическая значимость

Практическая значимость работы определяется влиянием реализованных механизмов на применимость библиотеки в прикладных сценариях:

- **Самоописание API** (программная интроспекция параметров, доступных возможностей и конфигурационных пресетов без инстанцирования тяжелых весов) критично для интеграции библиотеки с автодополнением в средах разработки, системами генерации MLOps-конфигураций и интерактивными блокнотами.
- **Потоковое обучение** расширяет область применения OmniAD на мониторинг высокочастотных сигналов в реальном времени (сетевые аномалии, телеметрия, логирование), где накопление батчей недопустимо по требованиям задержки.
- **Корректная сериализация** композитных моделей устраняет скрытые ошибки рассинхронизации сред развертывания, изолируя нативные веса от Python-состояния обертки.
- **Комплексное тестирование контрактов** гарантирует математическую эквивалентность оберток нативным реализациям, детерминированность вычислений и воспроизводимость сборки релизного пакета.

Задачи работы

1. Обеспечение прозрачности API библиотеки (discoverability)

1.1. Реализация модуля интроспекции параметров алгоритмов без их инстанцирования как единого источника метаданных для генератора `.pyi`-стабов и runtime-API.
1.2. Реализация функций `omniad.describe()` и `omniad.list_detectors()` для структурированного обзора и фильтрации зарегистрированных алгоритмов.
1.3. Проектирование декларативного реестра возможностей алгоритмов (`CAPABILITY_REGISTRY`) с автоматическим определением поддерживаемого функционала на основе миксинов.
1.4. Реализация единообразного представления объектов-детекторов (`__repr__`, `__str__`, `_repr_html_`).
1.5. Разработка механизма делегирования *capabilities* для составных (*composition*) адаптеров (TF-IDF, BERT), чей фактический набор возможностей зависит от внутреннего детектора.

2. Реализация поддержки потокового (streaming) обучения

2.1. Проектирование единой абстракции калибровки порога аномальности (`ThresholdStrategy`), обслуживающей батчевый и потоковый режимы без дублирования кода.
2.2. Реализация набора встроенных стратегий порогов (`quantile`, `sigma3`, `iqr`, `ewma`, `streaming_quantile`) и протокола регистрации пользовательских стратегий.
2.3. Реализация метода `partial_fit()` и миксина `IncrementalLearningMixin` на уровне базового класса детектора.
2.4. Реализация правила валидации единичного потокового сэмпла (`require_single_row`).
2.5. Разработка базового шаблона адаптера для потоковых библиотек (`BaseRiverAdapter`).
2.6. Реализация адаптера потокового алгоритма обнаружения аномалий `HalfSpaceTreesAdapter`.
2.7. Расширение системы опциональных зависимостей группой `streaming` и устранение дефектов валидации зависимостей в реестре.

3. Унификация обработки исключений на границе с внешними библиотеками

3.1. Приведение метода `predict_score()` к единому шаблону с автоматической трансляцией исключений сторонних библиотек в доменные ошибки OmniAD.

3.2. Реализация декоратора защиты публичных методов (`predict_map`, `predict_expected`, `get_feature_importances`), не входящих в стандартный цикл `fit`/`predict`.
3.3. Аудит и исправление дефектов модуля миксинов (недостающие импорты, стандартизация типов исключений).

4. Переработка механизма сериализации моделей

4.1. Разработка декларативного механизма исключения тяжелых и временных атрибутов (`_serialization_exclude`) с агрегацией по MRO.
4.2. Устранение дублирования состояния бэкенд-объектов (PyTorch, Transformers, внутренние детекторы, TF-IDF) при экспорте модели.
4.3. Приведение поведения параметра `save_weights` для трансформерных адаптеров в соответствие со спецификацией API.

5. Построение комплексной автоматизированной системы тестирования

5.1. Декомпозиция тестового набора по зонам ответственности (ядро, адаптеры уровня 1.5, конкретные алгоритмы, междоменные контракты, утилиты, реестр).
5.2. Разработка тестов жизненного цикла базового класса детектора и механизмов калибровки порогов.
5.3. Разработка parity-тестов, сверяющих результаты адаптеров с эталонными вызовами нативных библиотек.
5.4. Разработка тестов декларативного конвейера валидации входных данных.
5.5. Разработка тестов реестра метрик аномальности.
5.6. Разработка тестов потокового обучения и связанной с ним инфраструктуры.
5.7. Разработка междоменных контрактных тестов на достоверность заявленных *capabilities*.
5.8. Разработка тестов базовых шаблонов адаптеров (sklearn, torch, river, transformers, composition).
5.9. Разработка тестов изоляции и трансляции бэкенд-исключений.
5.10. Разработка тестов сохранения и загрузки моделей в ZIP-контейнеры.
5.11. Разработка тестов фабрики `get_detector()` и разрешения зависимостей.
5.12. Разработка тестов вспомогательных утилит нарезки временных рядов.
5.13. Интеграция проверки сборки дистрибутива (`wheel smoke test`) в CI-конвейер.
5.14. Разработка изолированного блока тестов визуализации результатов.

6. Разработка архитектурной спецификации подсистемы бенчмаркинга

Подготовка технической спецификации, регламентирующей протоколы замера накладных расходов оберток относительно нативных библиотек (*native baseline*), формат хранения артефактов и воспроизводимое представление метрик.

7. Реализация подсистемы бенчмаркинга согласно спецификации

7.1. Реализация контракта *native baseline* с контролем изоляции от внутренних модулей библиотеки.
7.2. Реализация `DatasetBundle` и протоколов разметки train/test по доменам.
7.3. Формирование синхронизированных реестров алгоритмов и датасетов для бенчмаркинга.

7.4. Реализация измерительного движка для сценариев *throughput* и *latency*.
7.5. Реализация раздельного профилирования потребления RAM и VRAM.
7.6. Реализация консольной точки входа для запуска измерений.
7.7. Реализация генерации отчетов (CSV/Markdown) и дифференциального анализа прогонов.
7.8. Реализация механизма контрольных точек для прерываемых длительных измерений.
7.9. Интеграция легковесных smoke-тестов бенчмарк-системы в CI.
7.10. Эмпирическая калибровка числовых порогов накладных расходов.

Список источников и научно-технических материалов

1. **Guha, S., Mishra, N., Roy, G., Schrijvers, O.** Robust Random Cut Forest Based Anomaly Detection on Streams // Proceedings of the 33rd International Conference on Machine Learning (ICML), PMLR 48:2712–2721, 2016.
   Ссылка: https://proceedings.mlr.press/v48/guha16.html
   Обзор материала: В статье представлен алгоритм Robust Random Cut Forest (RRCF) — потоковый ансамбль изолирующих деревьев для поиска аномалий во временных рядах и потоках данных. Работа решает фундаментальную задачу динамического обновления древовидных структур: авторы предлагают алгоритмы вставки и удаления точек из случайного дерева за время $O(\log n)$ с гарантией сохранения случайности структуры без полного переобучения на всем окне данных. Показано, как потоковая модель оценивает скор аномальности на лету через концепцию смещения глубины (*isolation score*), не требуя накопления исторического пакета данных в оперативной памяти.
   Зачем это в работе: Теоретическое обоснование принципов работы потоковых древовидных детекторов и концепции обновления ансамбля по одному входящему наблюдению (`partial_fit`), реализованной в `HalfSpaceTreesAdapter` (задачи 2.3 и 2.6). Механизм динамического обновления структуры объясняет, почему адаптер не накапливает матрицу наблюдений в памяти, обновляя внутреннее состояние пошагово.
2. **Montiel, J., Halford, M., Mastelini, S. M., et al.** River: machine learning for streaming data in Python // Journal of Machine Learning Research (JMLR), 22(110):1–8, 2021.
   Ссылка: https://jmlr.org/papers/v22/20-1380.html
   Обзор материала: Публикация посвящена архитектуре библиотеки River, созданной для потокового машинного обучения на языке Python. В статье обосновывается необходимость выделения специфического интерфейсного контракта для инкрементальных моделей: вместо пакетных матричных операций `fit(X)` / `predict(X)` вводится поэлементный протокол `learn_one(x)` / `predict_one(x)`. Обработка данных опирается на представление единичного объекта в виде словаря признаков, что обеспечивает гибкость при изменении структуры входного потока. Отдельное внимание уделено аппарату потоковой вычислительной статистики (инкрементальные средние, экспоненциальное сглаживание, скользящие квантили), функционирующему в условиях однопроходной обработки без материализации истории наблюдений в памяти.
   Зачем это в работе: Анализ архитектурных соглашений River определил проектирование базового шаблона `BaseRiverAdapter` (задача 2.5), где пакетное обучение декомпозировано в цикл вызовов потоковых примитивов. Метод `_row_to_dict()` обеспечивает преобразование векторов NumPy в словарный формат River, а структура `river.stats.Quantile` послужила ядром потоковой стратегии `StreamingQuantileThreshold` (задача 2.2).
3. **Zaharia, M., Chen, A., Davidson, A., et al.** Accelerating the Machine Learning Lifecycle with MLflow // IEEE Data Engineering Bulletin, 41(4):39–45, 2018.
   Ссылка: http://sites.computer.org/debull/A18dec/p39.pdf
   Обзор материала: В статье рассматривается проблема фрагментации жизненного цикла моделей машинного обучения при использовании гетерогенных вычислительных сред (scikit-learn, PyTorch, TensorFlow). Авторы формулируют концепцию абстракции артефакта модели (MLflow Models) как стандартизированного контейнера с декларативным описанием конфигурации (метаданные, программные зависимости, окружение) и изолированным хранением весов в их нативных форматах (так называемые *flavors*). Данный подход позволяет абстрагироваться от деталей внутреннего

представления библиотек при развертывании, устраняя риски нестабильности стандартной бинарной сериализации языка (в частности, Python `pickle`).
Зачем это в работе: Концепция послужила архитектурным ориентиром при переработке формата сохранения моделей OmniAD в виде изолированных ZIP-контейнеров (задача 4). Контейнер разделен на метаданные конфигурации (`metadata.json`), состояние легковесных параметров адаптера (`attributes.pkl`) и изолированный каталог `/backend`, куда делегируется сериализация нативными механизмами библиотек (`torch.save`, `joblib.dump`). Свойство `_serialization_exclude` исключает дублирование весов в основном архиве.

4. **Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P. F.** Producing Wrong Data Without Doing Anything Obviously Wrong! // Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 265–276, 2009.
Ссылка: https://users.cs.northwestern.edu/~robby/courses/322-2013-spring/mytkowicz-wrong-data.pdf
Обзор материала: Экспериментальное исследование феномена систематического смещения измерений (*measurement bias*) в компьютерных системах. Авторы показывают, что неконтролируемые параметры экспериментальной среды — размер переменных окружения (влияющий на выравнивание стека), порядок компоновки бинарных файлов и фиксированное чередование бенчмарков — могут искажать замеры времени выполнения до 300%, приводя к ложным выводам об эффективности оптимизаций. В качестве обязательного методологического требования предлагается изоляция среды выполнения и статистическая рандомизация условий запуска для устранения аппаратных корреляций.
Зачем это в работе: Исследование обосновывает методологические решения архитектурной спецификации бенчмаркинга (задачи 6 и 7.4): изоляцию замеров через независимые подпроцессы `subprocess.run` (предотвращающую загрязнение памяти и кэша CUDA) и псевдослучайную рандомизацию порядка запуска сценариев `native` и `omniad` для устранения асимметрии из-за теплового троттлинга процессора.
5. **Han, S., Hu, X., Huang, H., Jiang, M., Zhao, Y.** ADBench: Anomaly Detection Benchmark // Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 35:32142–32159, 2022.
Ссылка: https://arxiv.org/abs/2206.09426
Обзор материала: Крупнейший открытый бенчмарк алгоритмов обнаружения аномалий, охватывающий 57 табличных наборов данных (включая классические выборки ODDS: Thyroid, Shuttle и др.), а также домены NLP и CV. В работе детально исследована методология корректного измерения производительности: оценка качества ранжирования через PR-AUC и ROC-AUC в условиях сильного дисбаланса классов, устойчивость алгоритмов к уровню загрязнения выборки шумом (*contamination*), а также профилирование вычислительных накладных расходов (время обучения, задержка инференса и потребление памяти).
Зачем это в работе: Служит методологической базой подсистемы бенчмаркинга качества и накладных расходов (задачи 6, 7.2–7.5). Обосновывает применение метрик PR-AUC и ROC-AUC в модуле `safe_auc`, включение стандартных датасетов ODDS в `benchmarks/data/registry.py` и раздельное измерение пропускной способности (*throughput*) и поштучной задержки (*latency*).
6. **Lones, M. A.** How to Avoid Machine Learning Pitfalls: A Guide for Academic Researchers // arXiv preprint arXiv:2108.02497, 2021.
Ссылка: https://arxiv.org/abs/2108.02497
Обзор материала: Систематизированное руководство по предотвращению скрытых

методологических ошибок на всех стадиях жизненного цикла ML-систем. Рассматриваются проблемы утечки данных (*data leakage*) при предобработке, нарушение структуры зависимости данных при случайном перемешивании временных последовательностей, неконтролируемые источники стохастичности (инициализация генераторов псевдослучайных чисел, многопоточный недетерминизм библиотек) и чрезмерное доверие единичным точечным метрикам без верификации стабильности результатов. *Зачем это в работе:* Принципы руководства положены в основу контрактных тестов библиотеки (задачи 5.1–5.13): тесты детерминированности по `random_state` гарантируют воспроизводимость скоров, а обязательный хронологический протокол разбиения без перемешивания в фикстурах `AllDomainDatasets` и утилите `create_windows` исключает заглядывание в будущее при валидации моделей временных рядов.